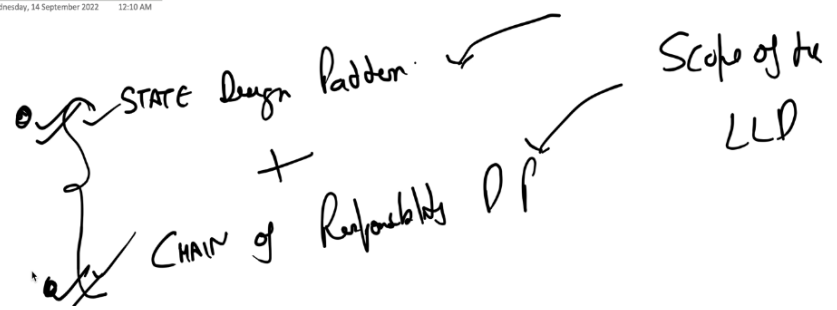
18. LLD of ATM

https://gitlab.com/shrayansh8/interviewcodingpractise/-/tree/main/src/main/java/com/conceptandcoding/LowLevelDesign/DesignATM?ref_type=heads

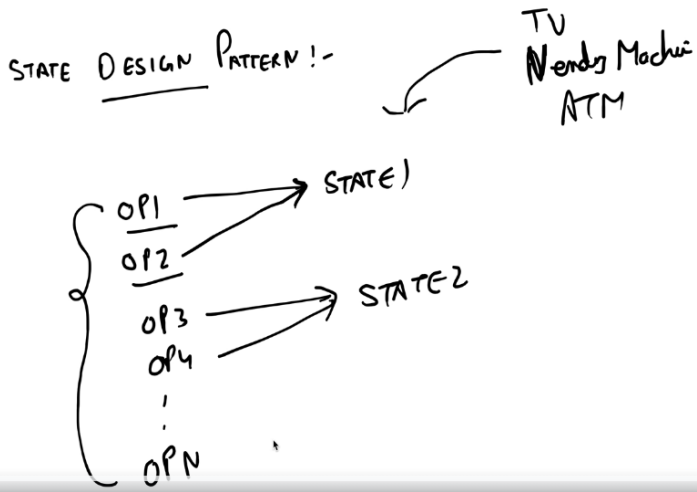
Design ATM

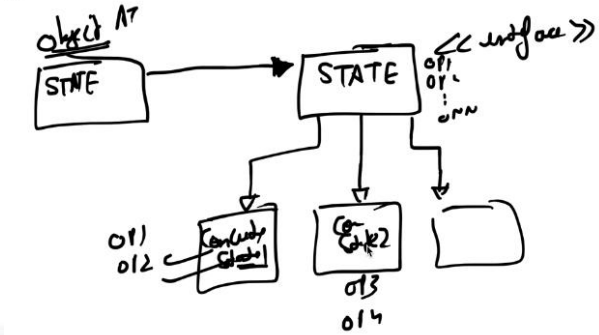
Wednesday, 14 September 2022 12:10 AM



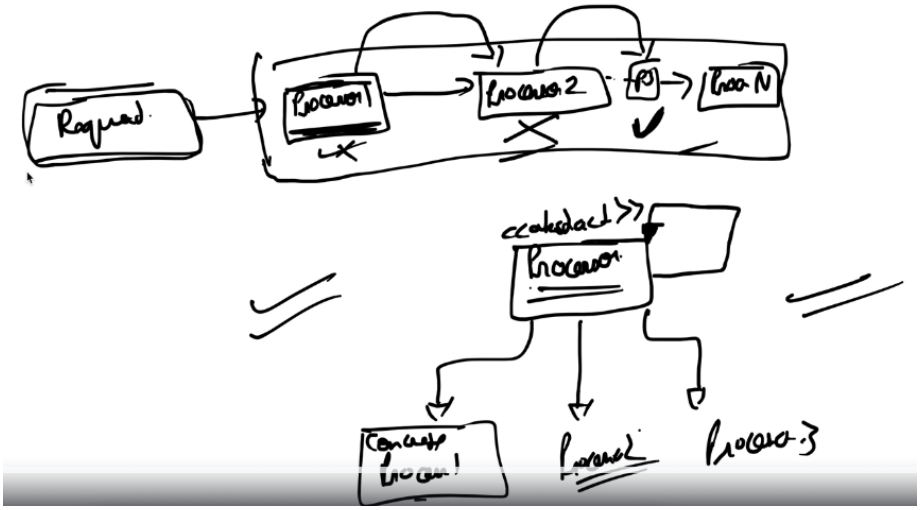
Recap of State design pattern and Chain of Responsibility

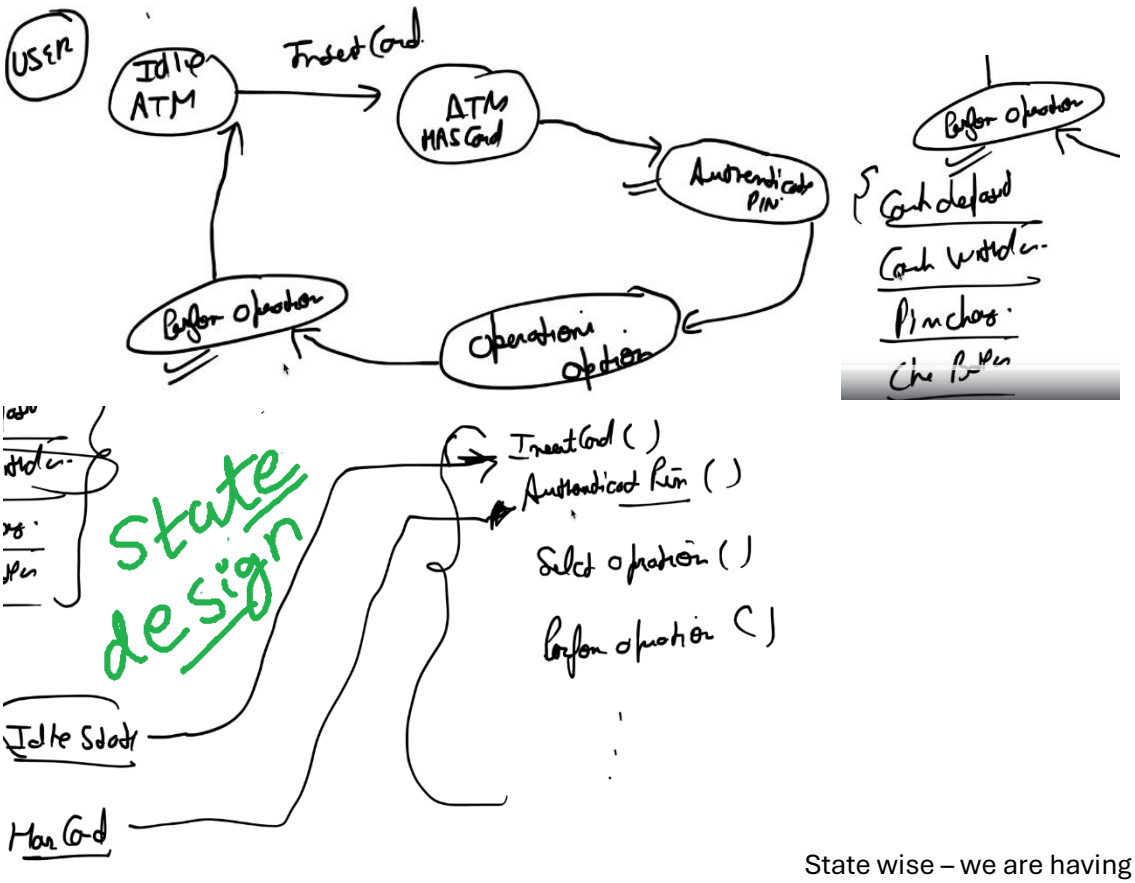
Wednesday, 14 September 2022 12:12 AM



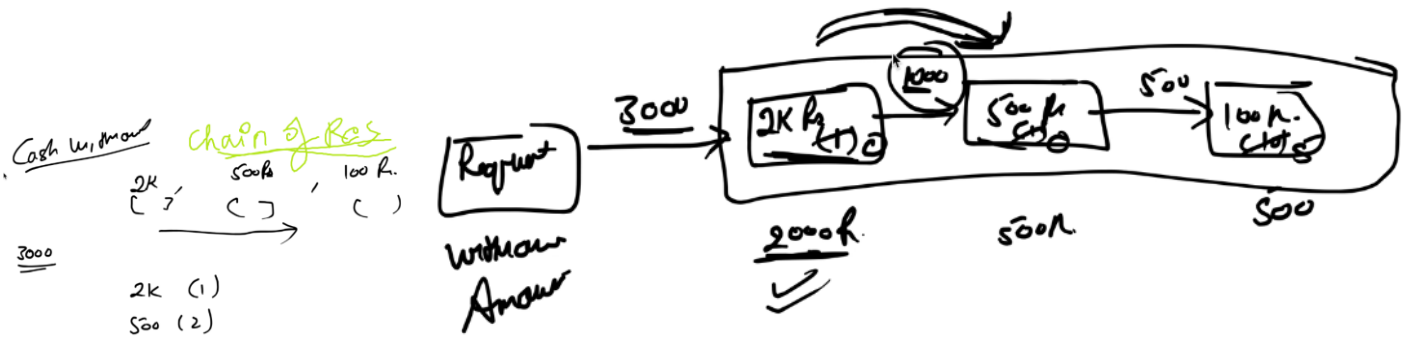


Chain of Responsibility





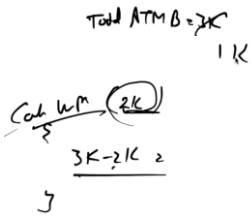
State wise – we are having specific operations



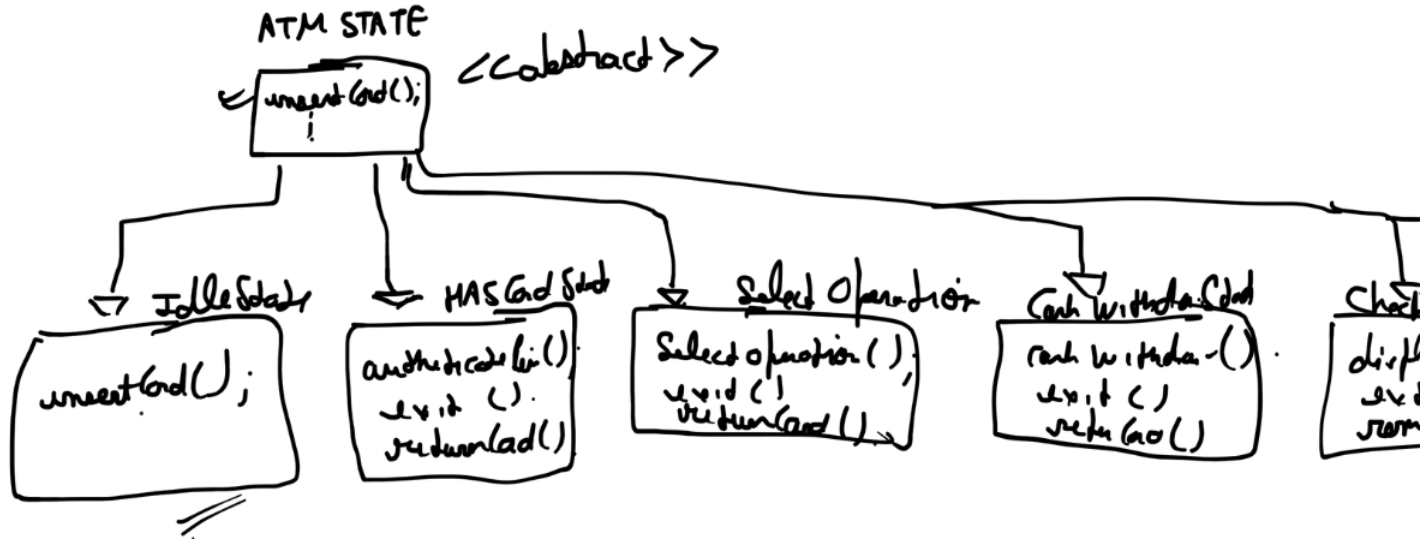
If request cannot be fulfilled, it is forwarded to the next processor - chain of responsibility (cash withdrawal)

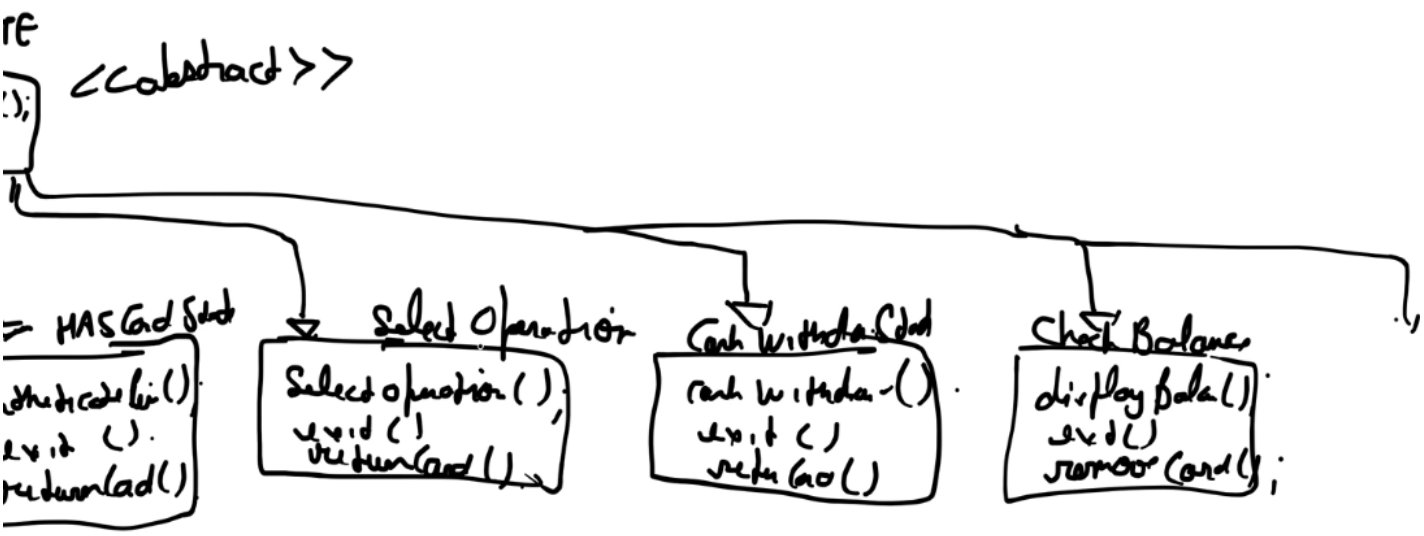
chain of responsibility (cash withdrawal) use case :

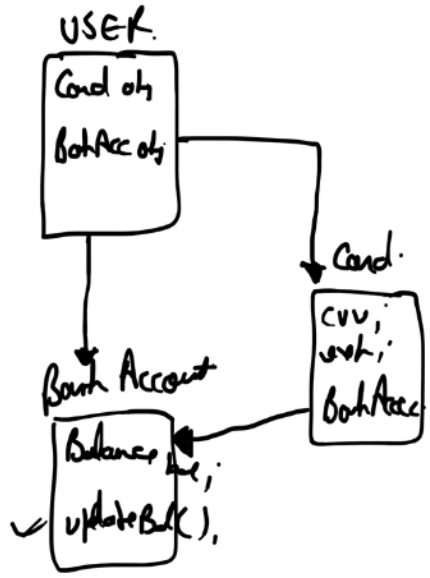
Can we just assume that when we do cash withdrawal method implementation, we can just simply reduce the amount. Let's say the total ATM balance is 3k and withdrawal request is 2k, then 1k is left
If the interviewer says that he is interested in - How you going to withdraw the amount? Then you need a chain of responsibility design pattern also

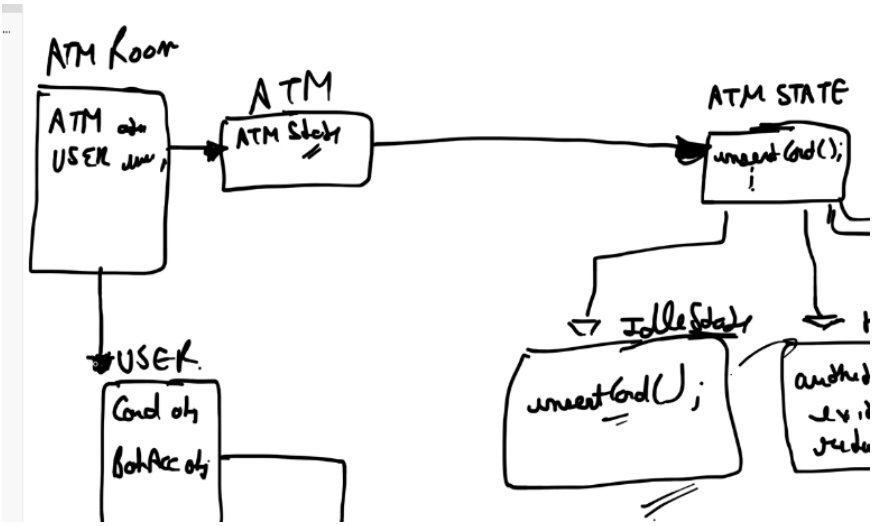


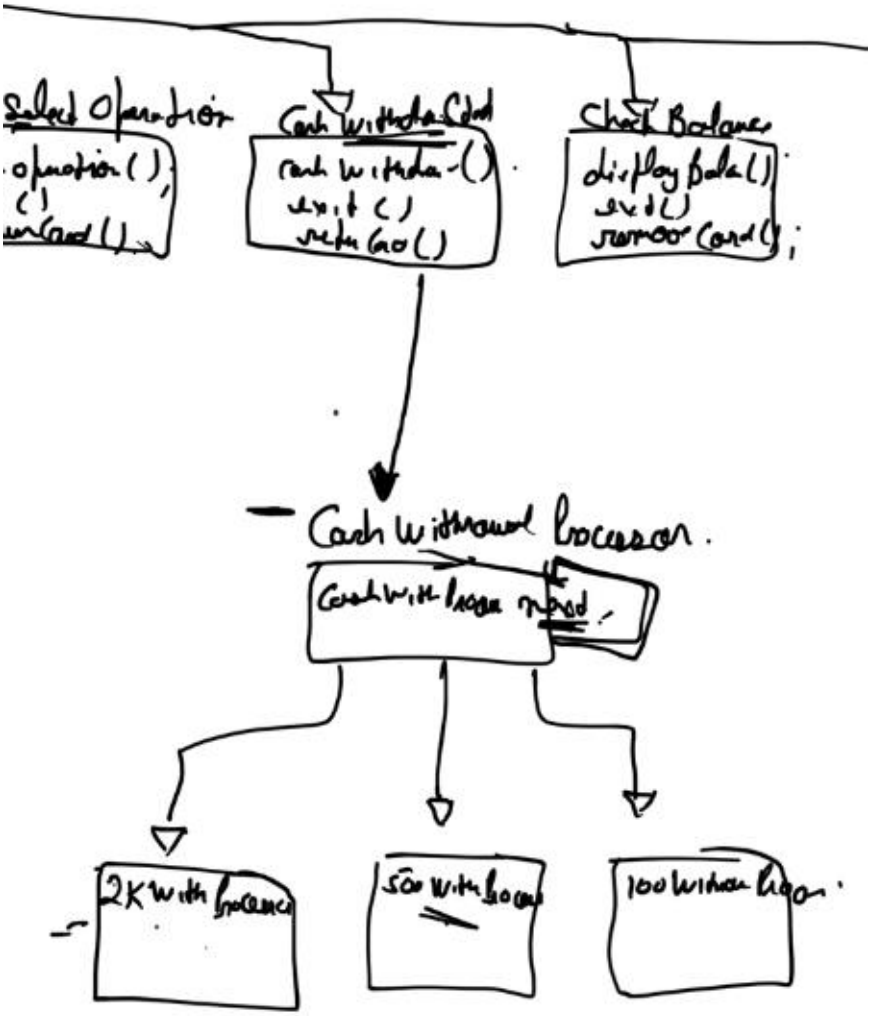
MVP – Minimum Value Product - minimum collection of features and behavior that are required

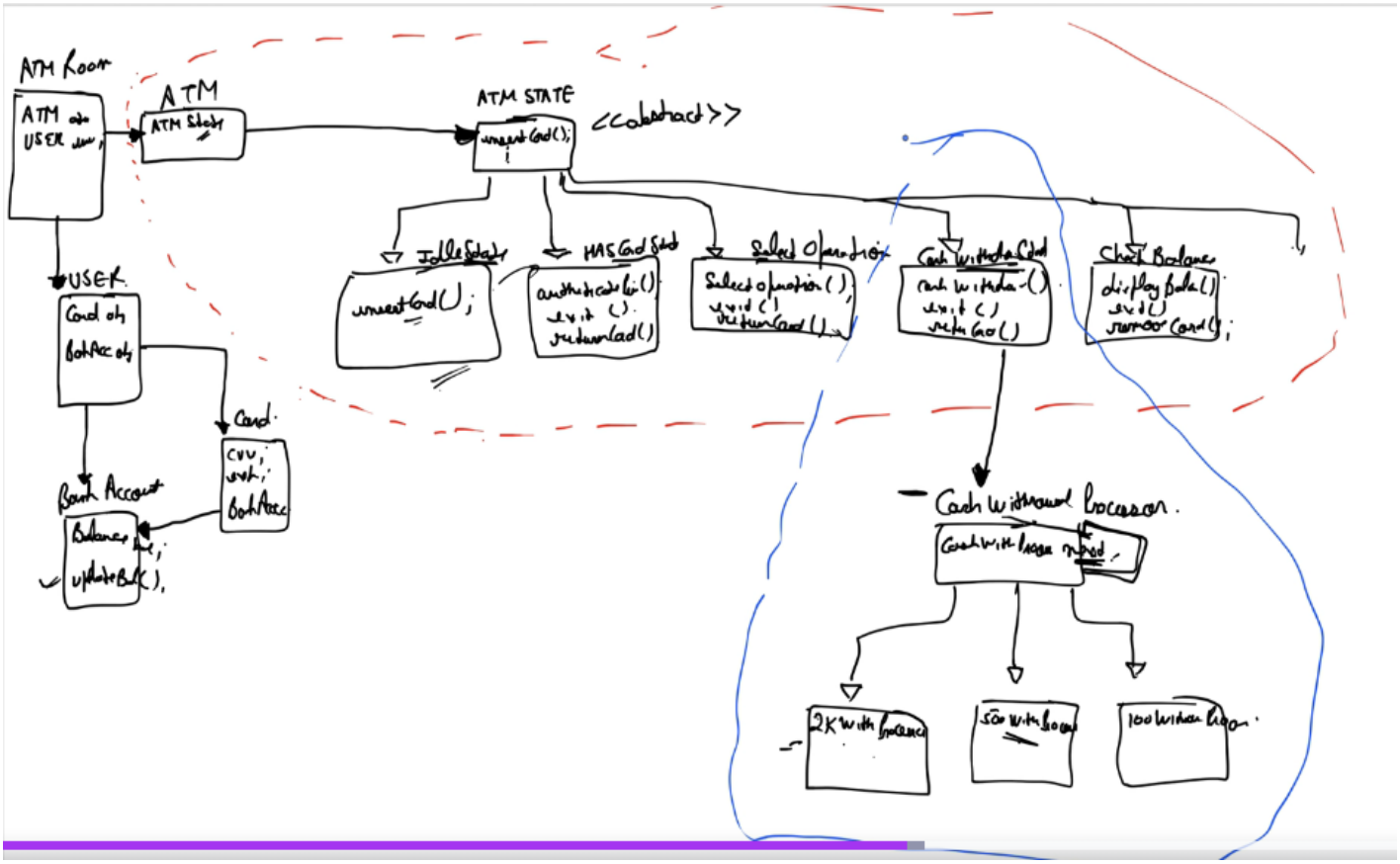


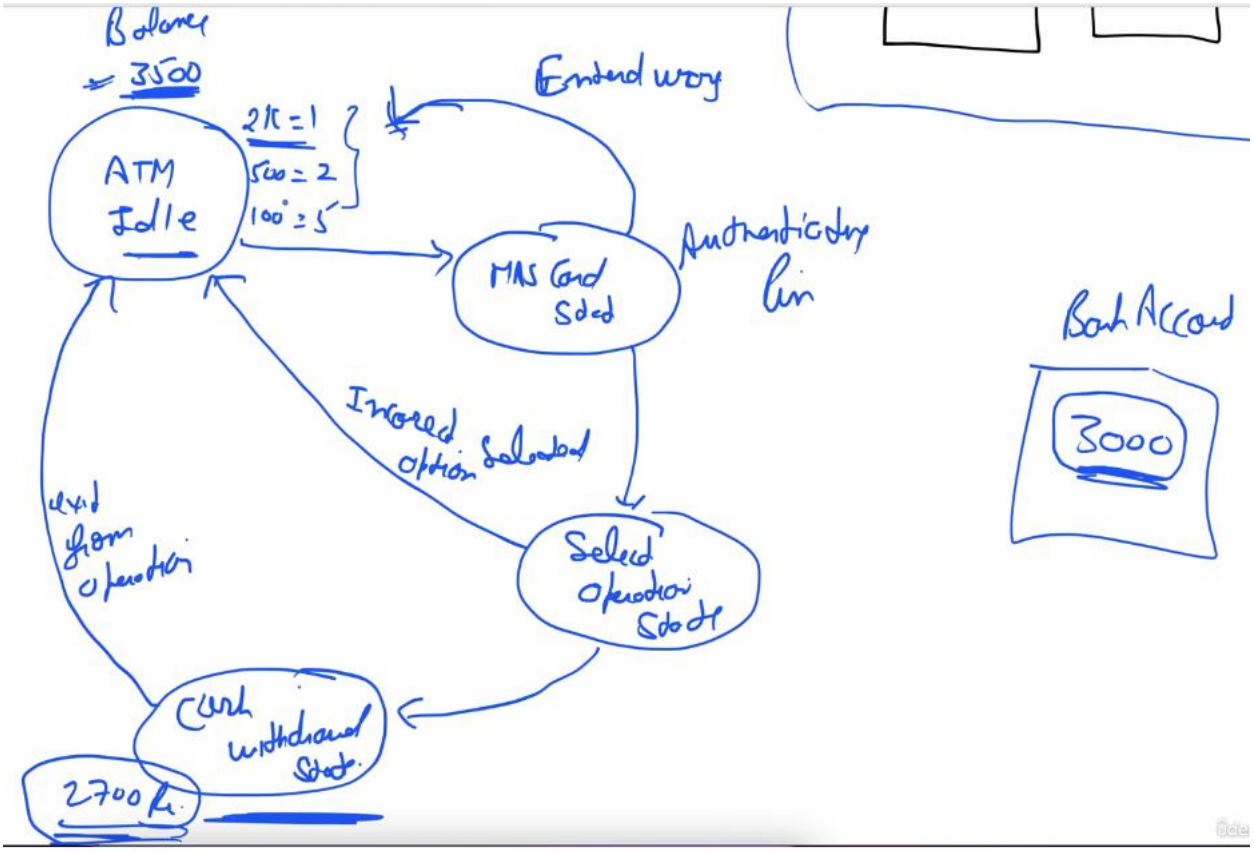


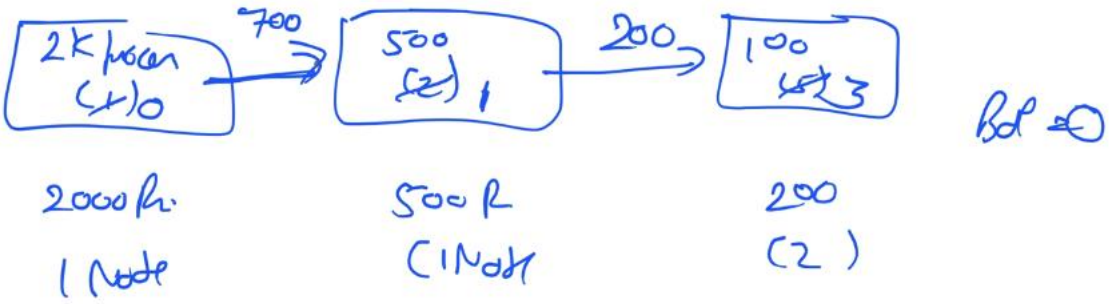





Select Operation
Cash Withdrawal
Check Balance
Cash Withdrawal Processor.
2K Withdrawal
500 Withdrawal
100 Withdrawal


ATM Room
ATM
ATM STATE
<<abstract>>
IdleState
HASCardState
Select Operation
Cash Withdrawal
Check Balance
USER
Card
Bank Account
Cash Withdrawal Processor.
2K Withdrawal
500 Withdrawal
100 Withdrawal



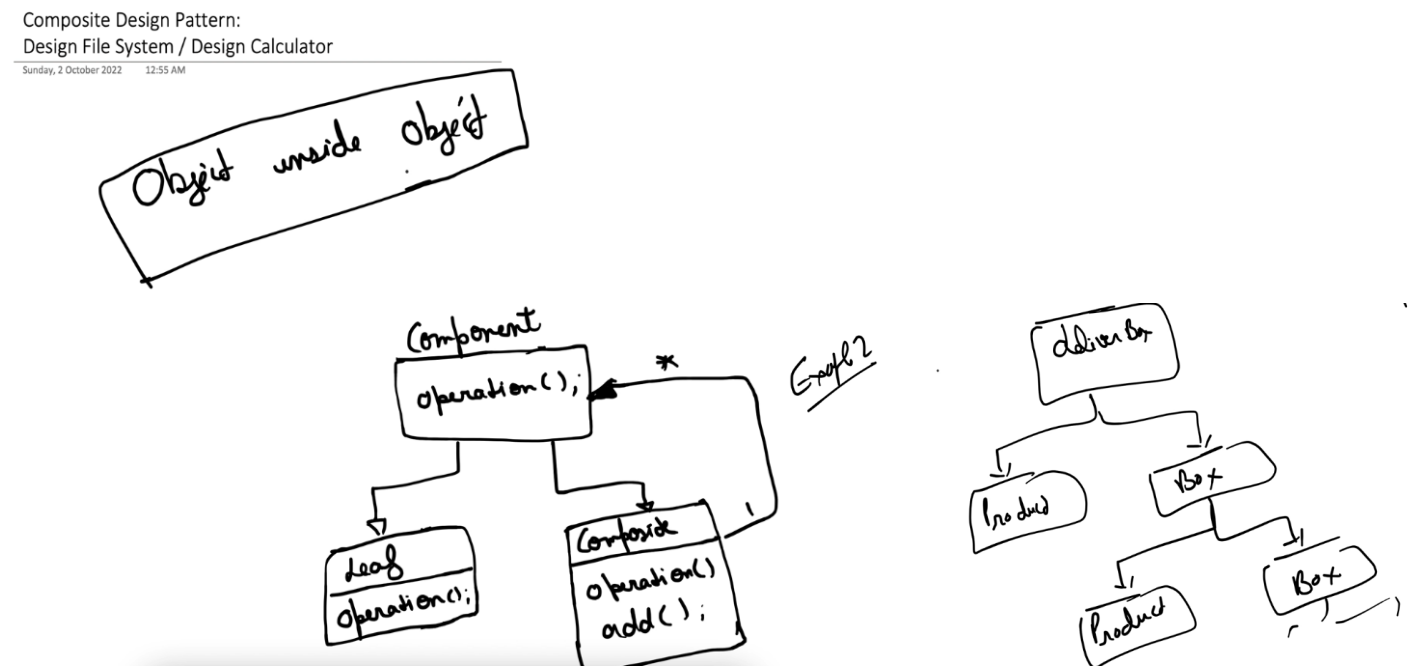


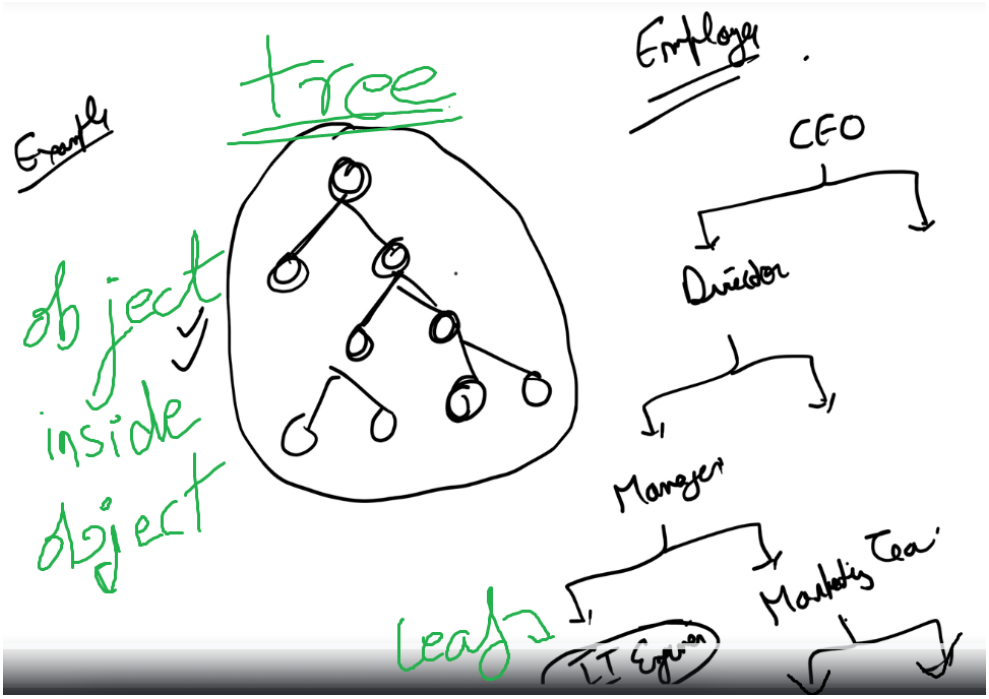

```
/Library/Java/JavaVirtualMachines/zu
Balance: 3500
2kNotes: 1
500Notes: 2
100Notes: 5
Card is inserted
enter your card pin number
Please select the Operation
CASH_WITHDRAWAL
BALANCE_CHECK
Please enter the Withdrawal Amount
Please collect your card
```

```
CASH_WITHDRAWAL
BALANCE_CHECK
Please enter the Withdrawal Amount
Please collect your card
Exit happens
Balance: 800
2kNotes: 0
500Notes: 1
100Notes: 3
```

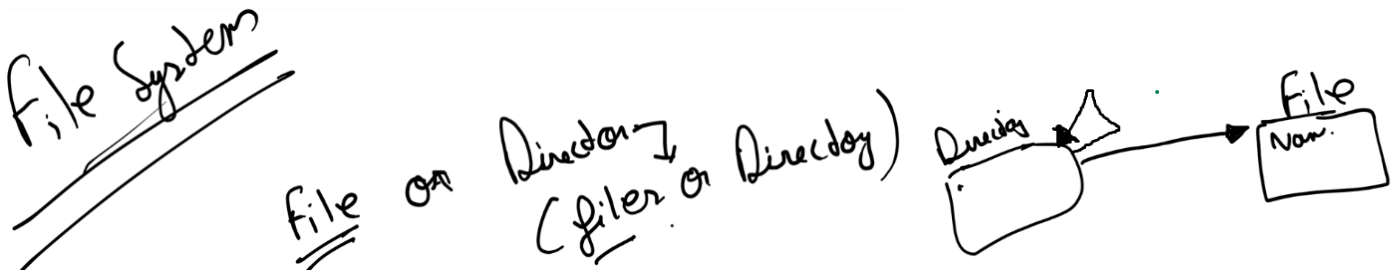
https://gitlab.com/shrayansh8/interviewcodingpractise/-/tree/main/src/main/java/com/conceptandcoding/LowLevelDesign/DesignATM?ref_type=heads

19. Composite Pattern (Structural Design Pattern)
https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/structuralpatterns/composite?ref_type=heads

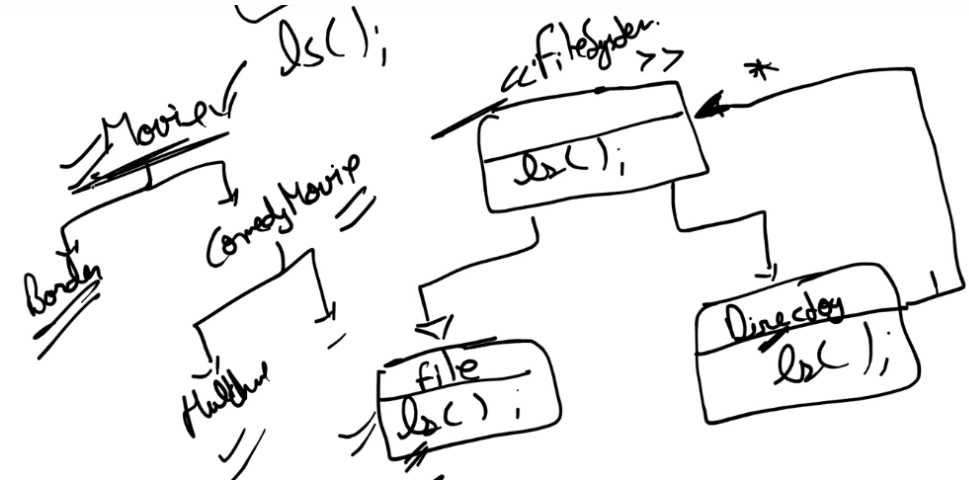




File system: Problem:



File system: Solution:



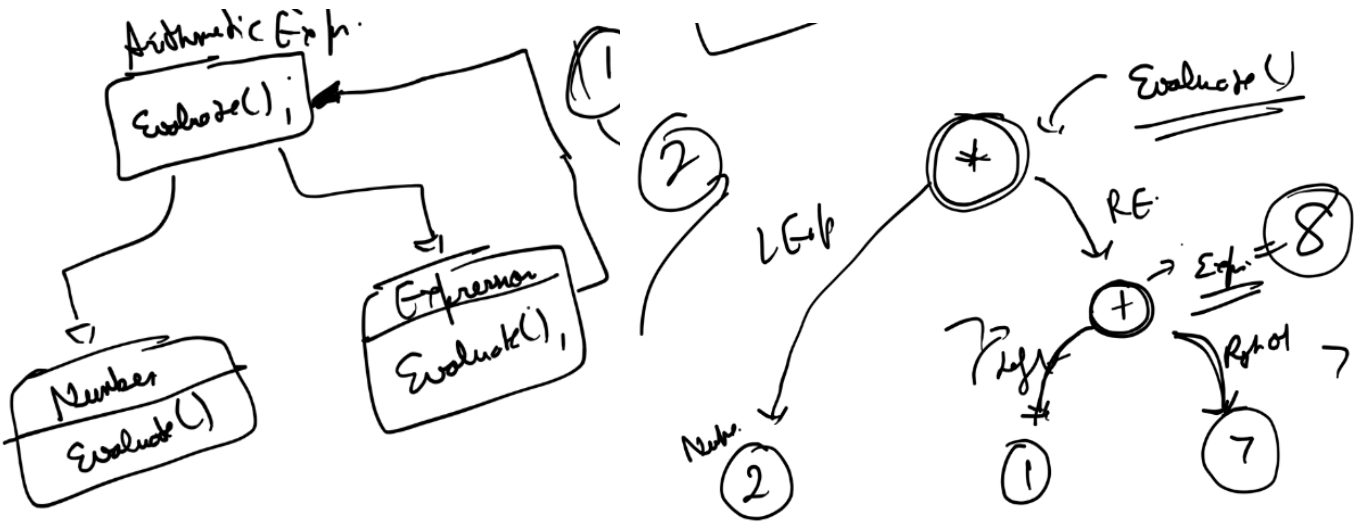

```java
public class Main {
    public static void main(String args[]){

        Directory movieDirectory = new Directory( name: "Movie");

         FileSystem border = new File( name: "Border");
         movieDirectory.add(border);

        Directory comedyMovieDirectory = new Directory( name: "ComedyMovie");
        File hulchul = new File( name: "Hulchul");
         comedyMovieDirectory.add(hulchul);
         movieDirectory.add(comedyMovieDirectory);

         movieDirectory.ls();

    }
}
```

```
Main
/Library/Java/JavaVirtualMachines/z
Directory name Movie
file name Border
Directory name ComedyMovie
file name Hulchul

Process finished with exit code 0
```

https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/structuralpatterns/composite/filestructure?ref_type=heads

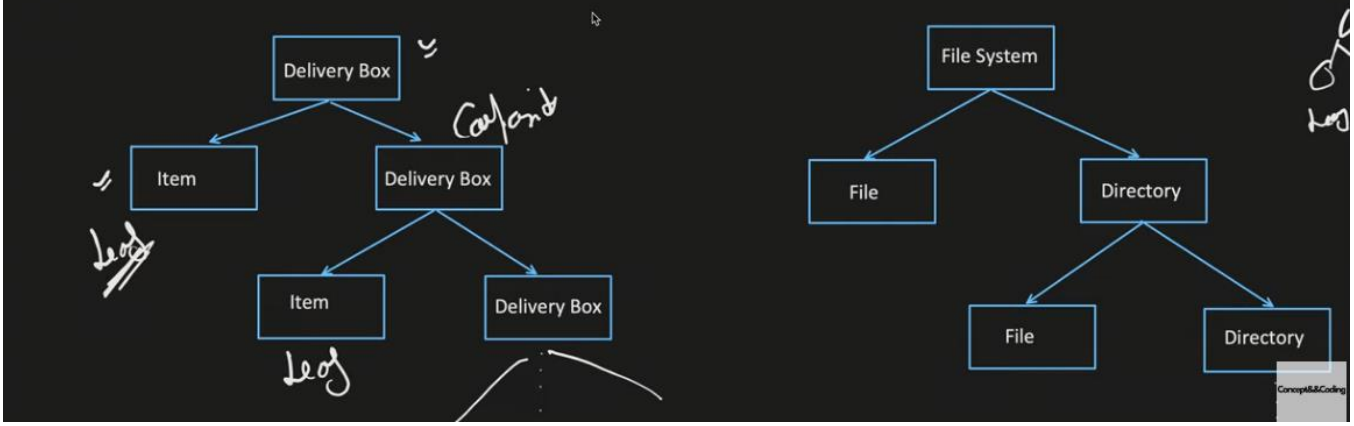
Calculator: Solution:

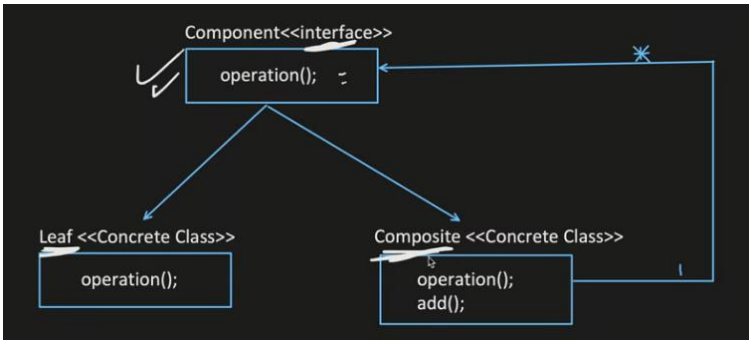
https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/structuralpatterns/composite/mathexpression?ref_type=heads



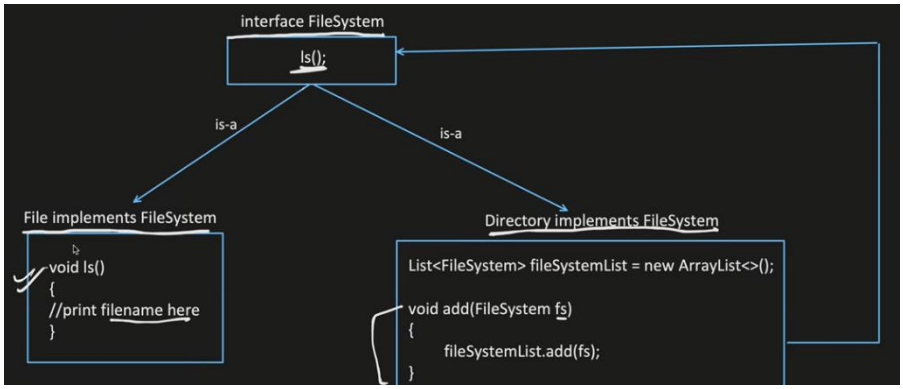
3. Composite Pattern:

This pattern helps in scenarios where we have OBJECT inside OBJECT (tree like structure)





Example:



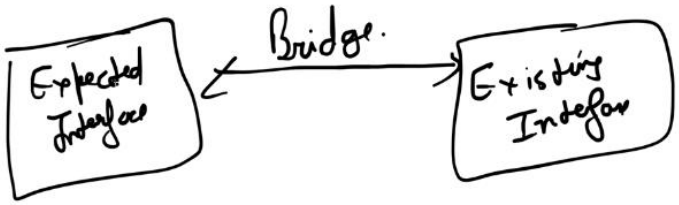

```
Directory implements FileSystem

List<FileSystem> fileSystemList = new ArrayList<>();

void add(FileSystem fs)
{
      fileSystemList.add(fs);
}

void ls()
{
      for(FileSystem fsObj : fileSystemList)
      {
            fsObj.ls();
      }
}
```

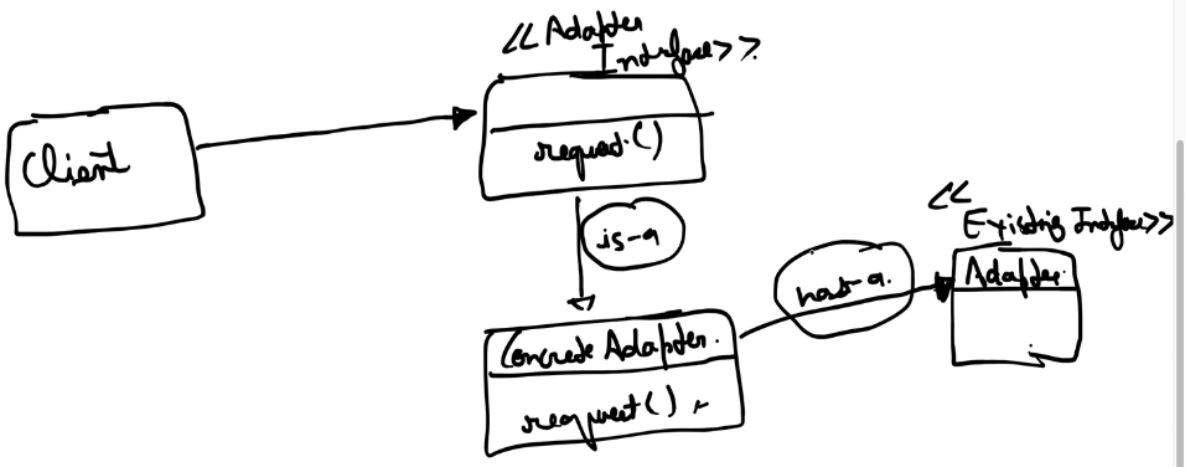
```
Directory parentDir= new Directory();
FileSystem fileObj1 = new File();

parentDir.add(fileObj1);

Directory childDir = new Directory();
FileSystem fileObj2 = new File();
childDir .add(fileObj2);

parentDir.add(childDir);

parentDir.ls();
```

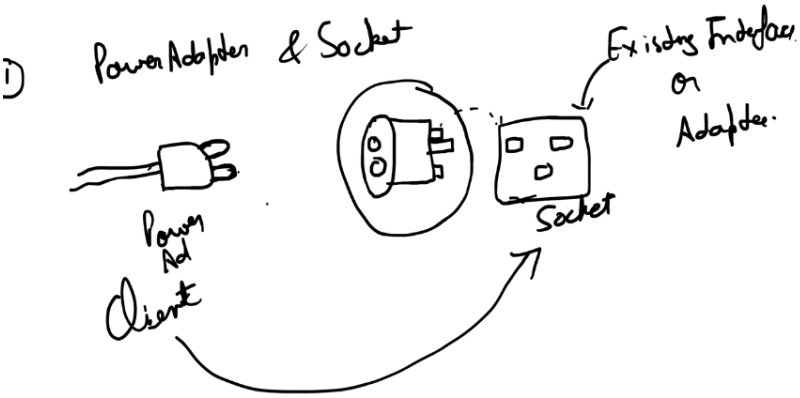
20. Adapter Pattern (Structural Design Pattern)

LLD: Adapter Design Pattern

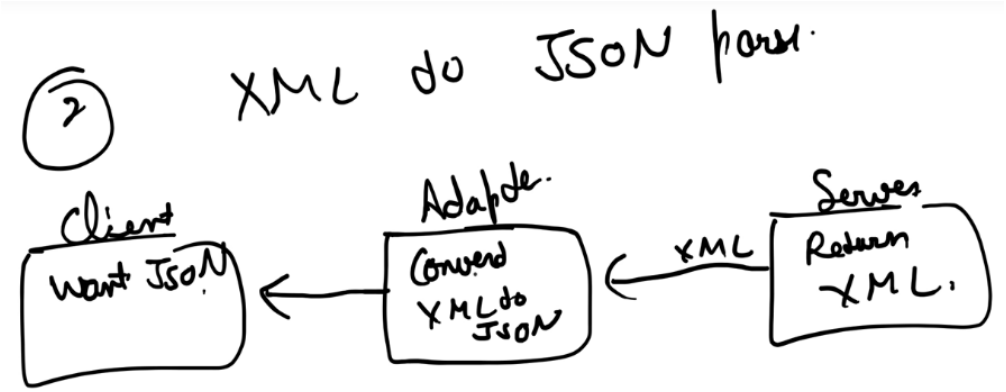
Saturday, 15 October 2022 10:08 PM





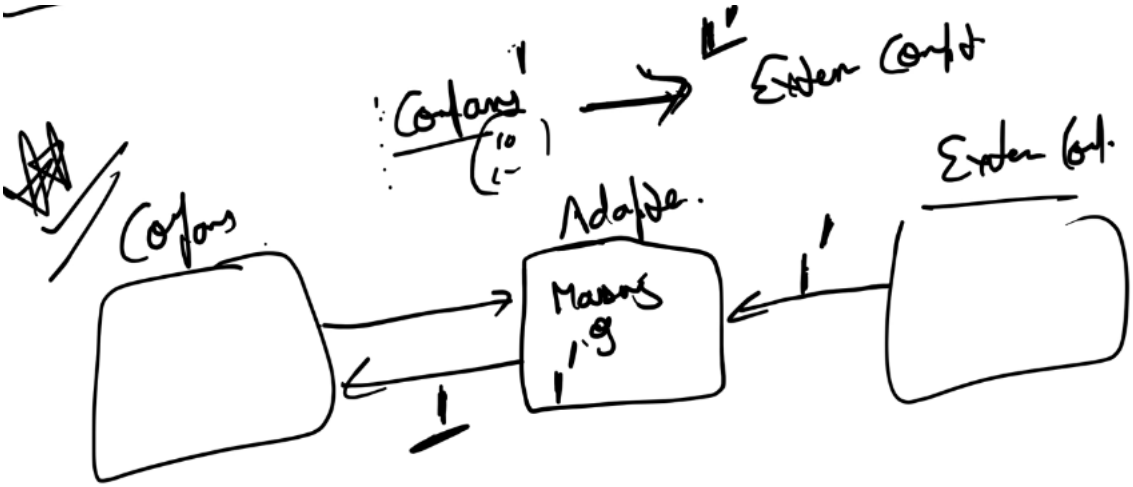


You cannot use a power adapter directly to this socket because they are not compatible with each other. adapter implements this interface – bridge.

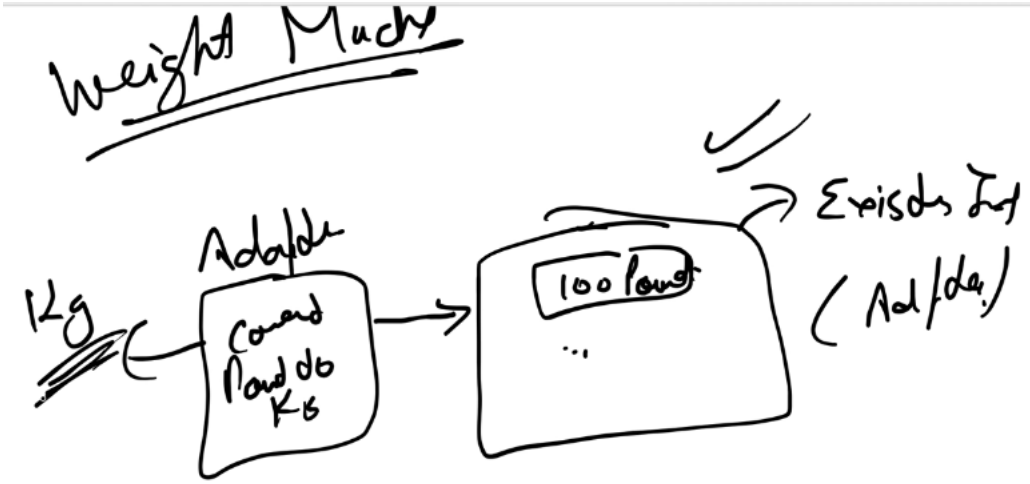


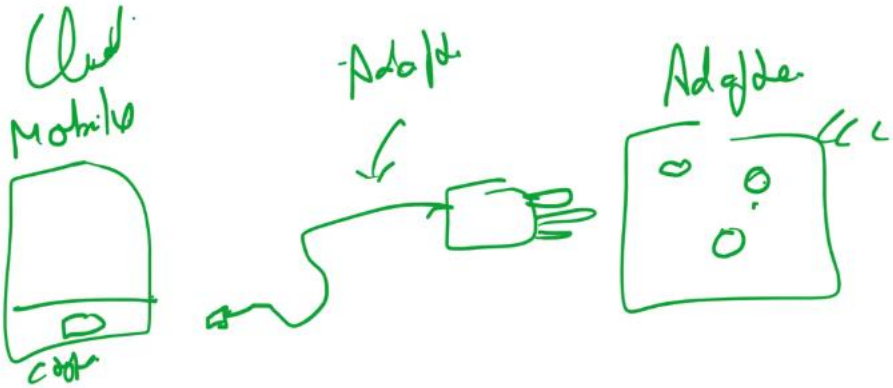
how you will take care of the use cases where you are getting a request or response, which is Different from what you wanted?

We will use the adapter design pattern.

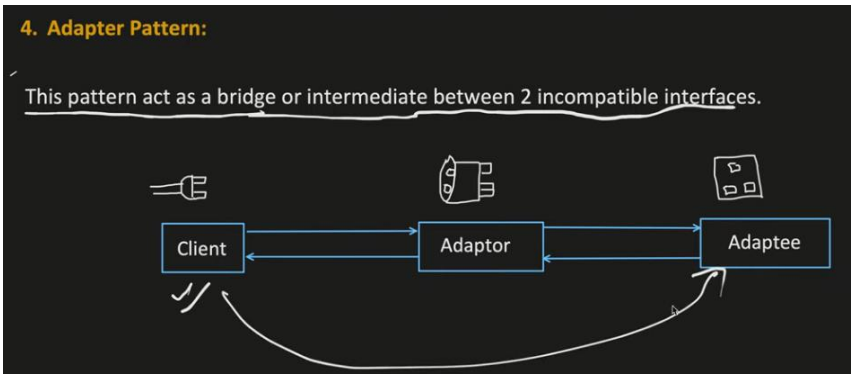


https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/structuralpatterns/adapter?ref_type=heads





client which wanted to access this adoptee, but client has one kind of structure, adoptee other kind of structure both are incompatible
client calls adapter, adapter calls adoptee. adoptee returns to the adapter - adapter changes the response whichever the client understands and return it . so it is a mediator now



WeightMachineAdaptor

tWeightInKg();

is-a

pl implements WeightMachineAdaptor

hine;

rImpl(WeightMachine machine)

achine;

has-a

interface WeightMachine

int getWeightInPounds();

is-a

WeightMachineImpl implements WeightMachine

```
int getWeightInPounds()
{
    return 30;
}
```

int getWeightInKg();

is-a

WeightMachineAdaptorImpl implements WeightMachineAdaptor

```
WeightMachine machine;

WeightMachineAdaptorImpl( WeightMachine machine)
{
    this.machine = machine;
}

int getWeightInKg()
{
   int weightInPound = machine.getWeightInPounds();
   return weightInPound * .45;
}
```

has-a

Client

```
WeightMachineAdaptor machineAdaptorObj = new WeightMachineAdaptorImpl(new WeightMachineImp
machineAdaptorObj .getWeightInKg();
```

interface WeightMachineAdaptor

int getWeightInKg();

is-a

WeightMachineAdaptorImpl implements WeightMachineAdaptor

interface WeightMachine

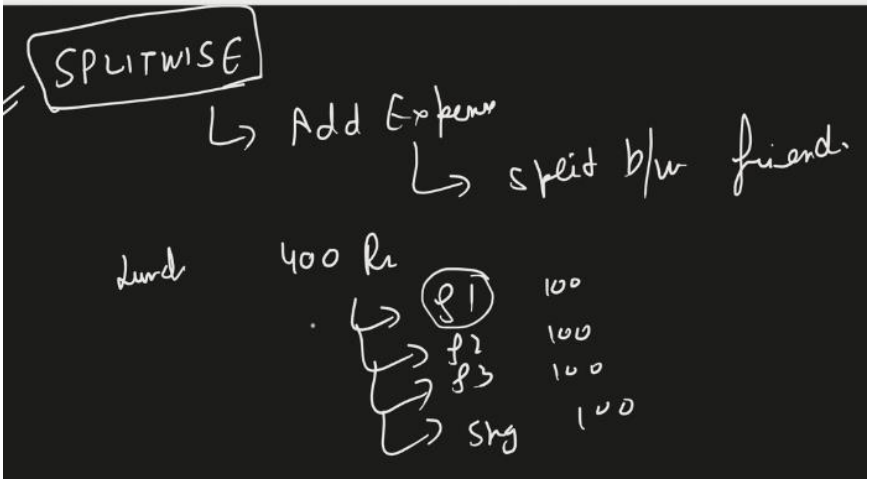
int getWeightInPounds();

is-a

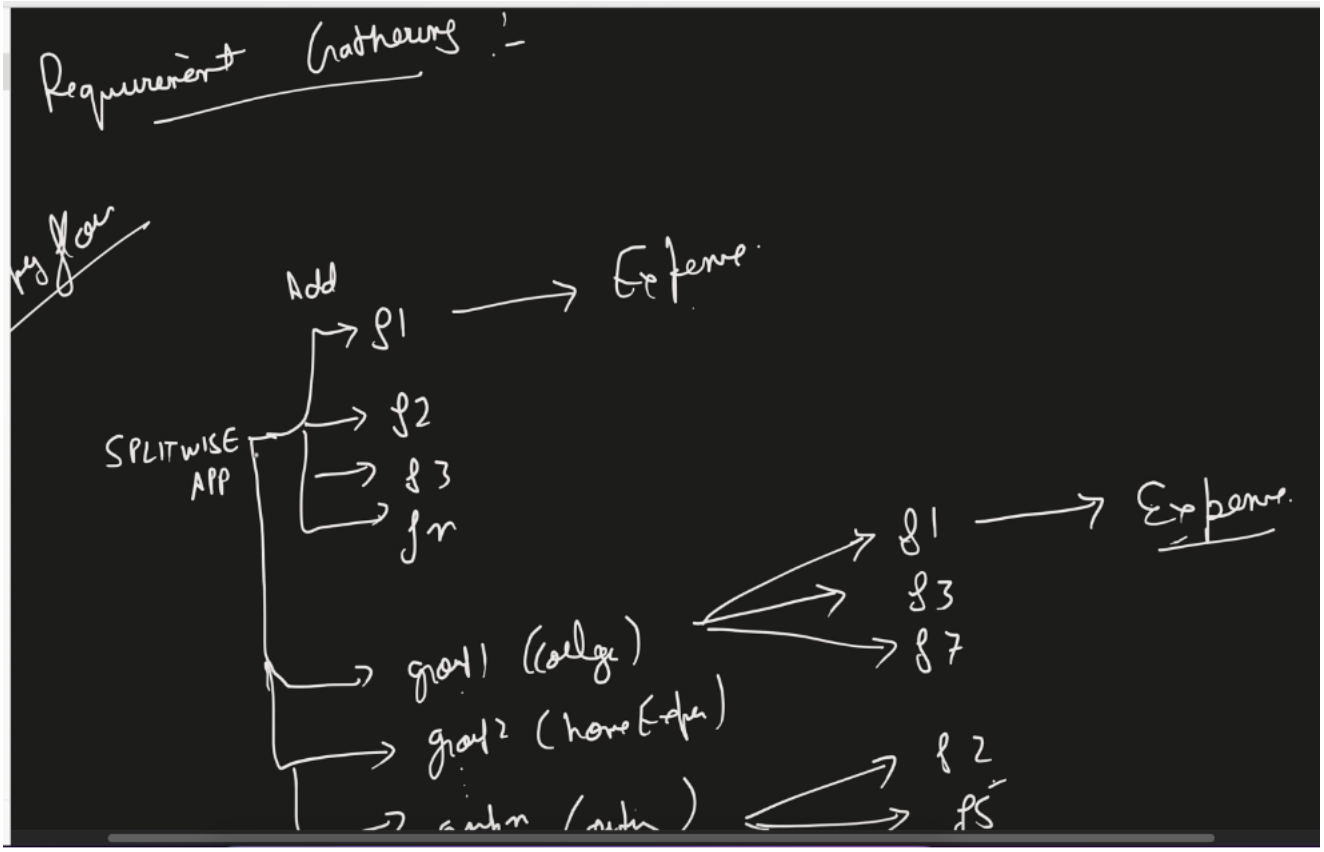
WeightMachineImpl implements W

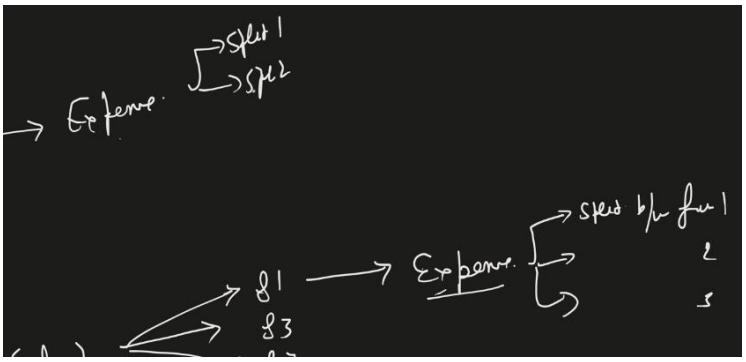
21. LLD of Split wise

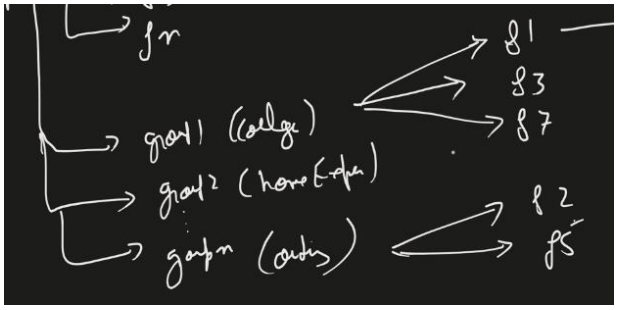
https://gitlab.com/shrayansh8/interviewcodingpractise/-/tree/main/src/main/java/com/conceptandcoding/LowLevelDesign/DesignSplitwise?ref_type=heads



Requirement gathering: (happy flow)







Requirements :-

1. Add friends Capability
2. Add / Manage Group
3. Add / Manage friends inside a group
4. Manage Expenses Inside a group / without a group
5. SPLIT Expenses Capability

5. SPLIT Expenses Capability
   - → Equal
   - → Unequal
   - → % wise Split

6. Balance Sheet of Each User.

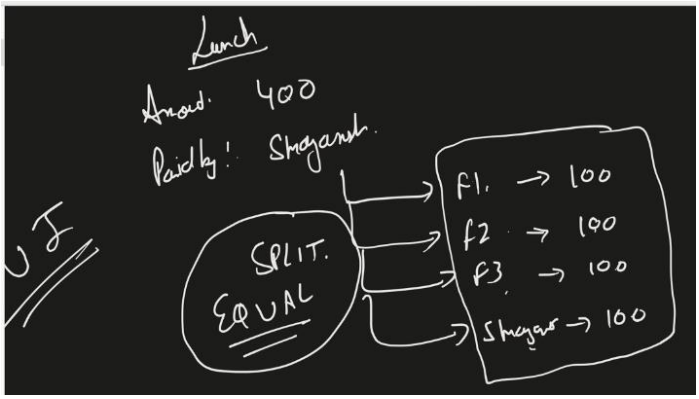
400 → 100, 100, 100, 100

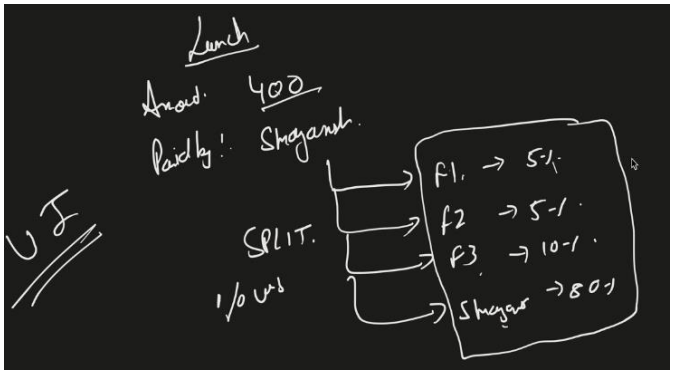
100	50	10 %
100	100	20 %
100	50	5 %
100	200	5 %

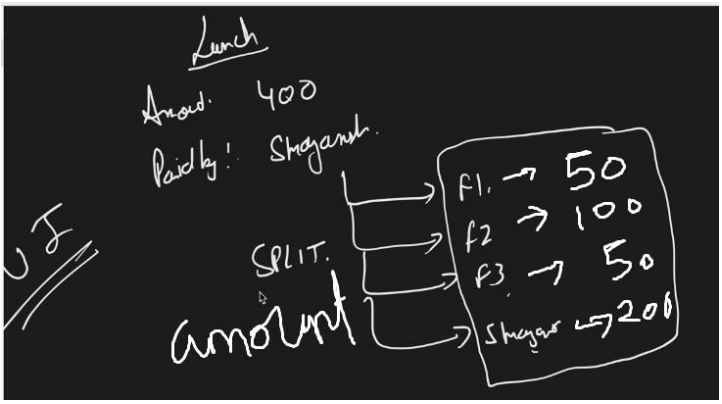
Step 2 :- Object Identification :-

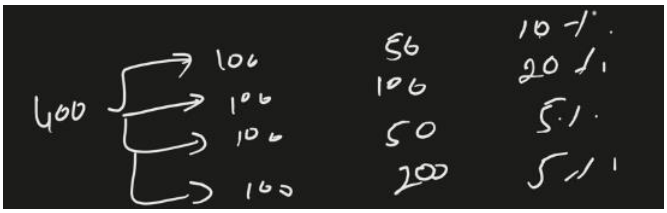
1. SPLITWISE
2. USER
3. GROUP
4. Expense
5. SPLIT ??
6. BalanceSheet

What happens when an expense is created?



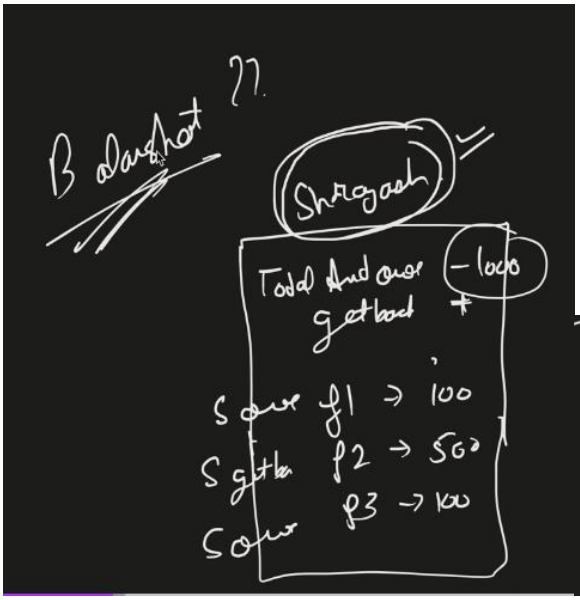


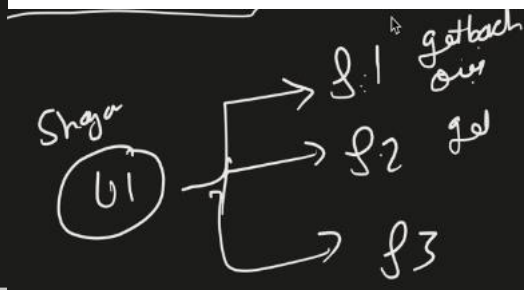




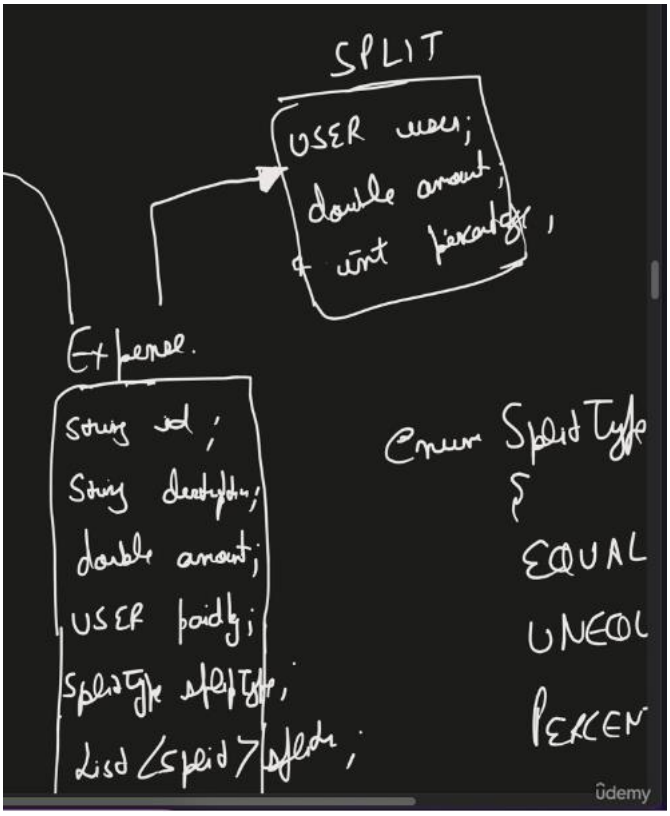
3 scenarios:

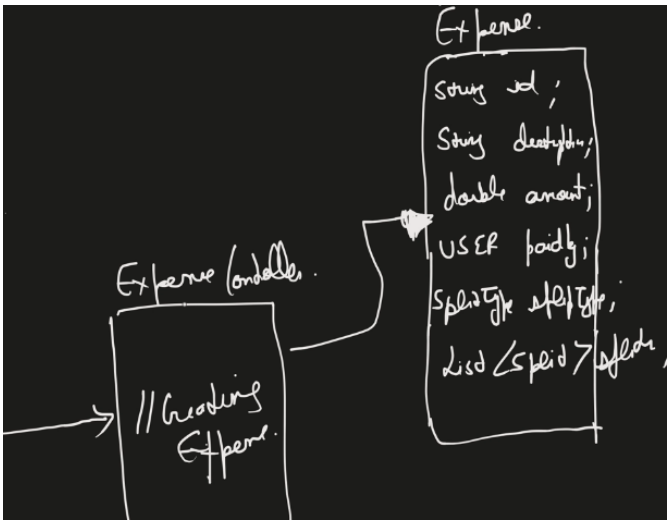
- Equal – amount equally split
- In the UI send the percentage, then server will compute the amount
- UI will compute the amount and then will send it

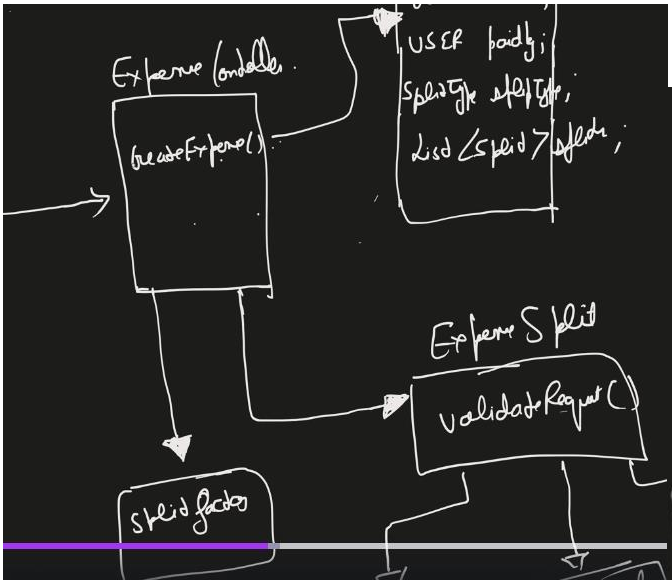


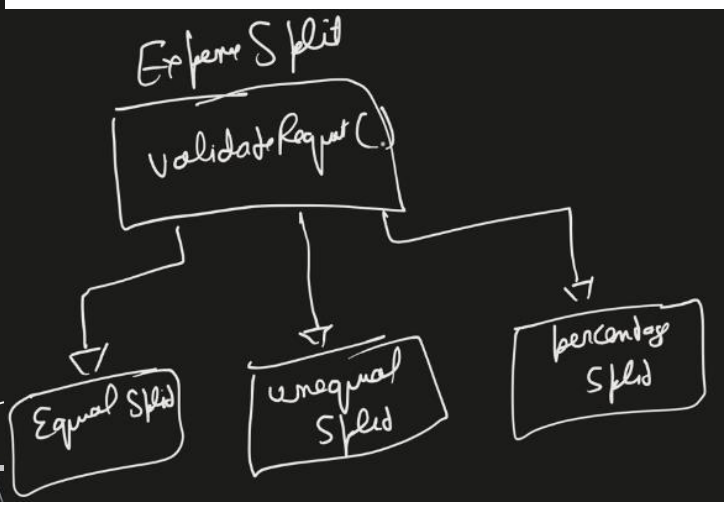


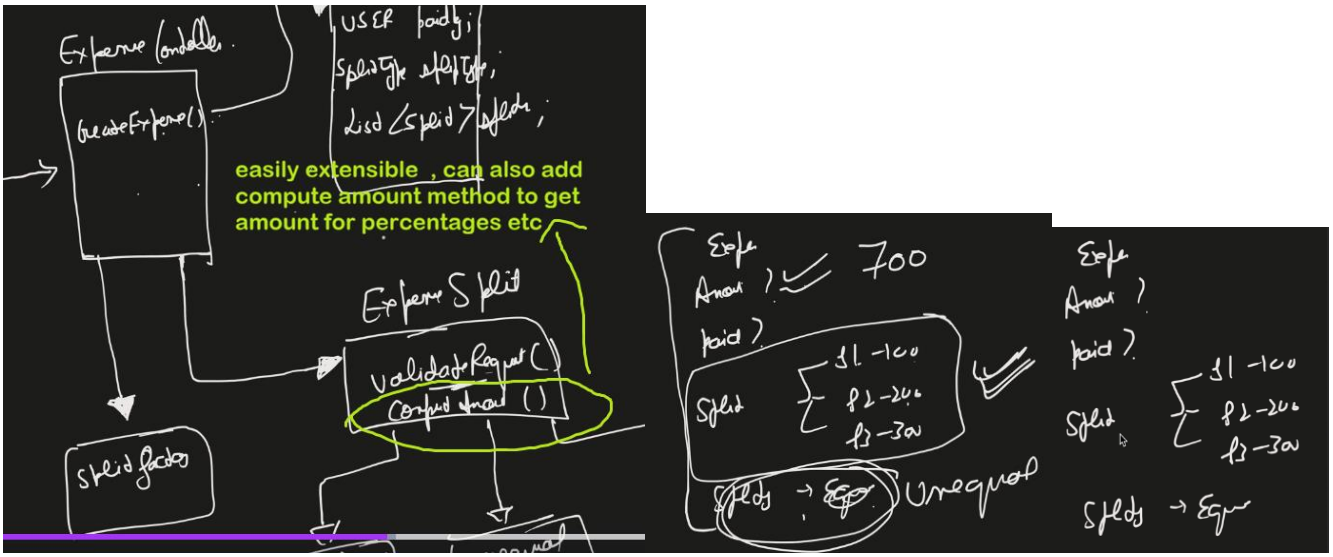
UML :

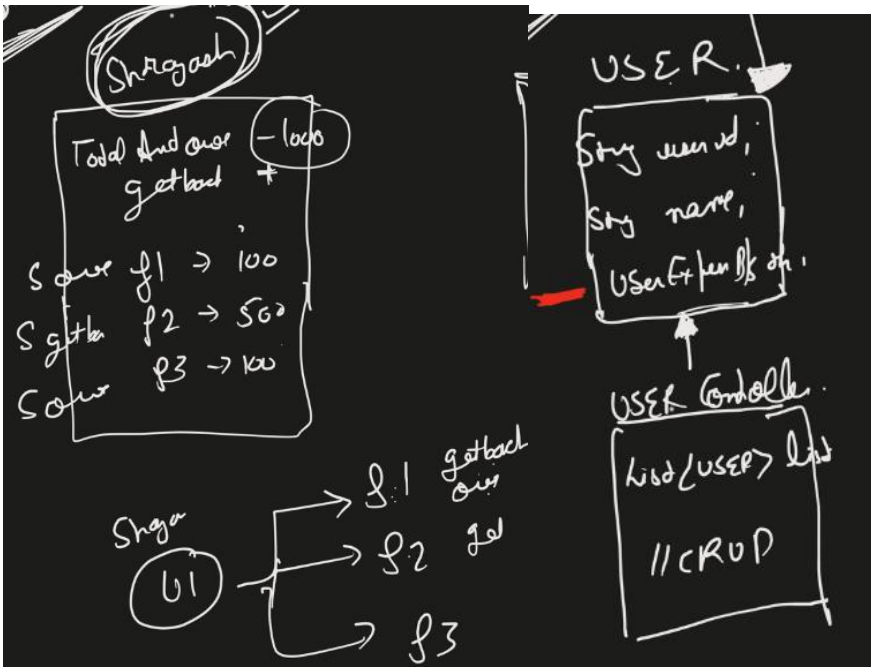

SPLIT
USER user;
double amount;
int percentage;
Expense.
String id;
String description;
double amount;
USER paidBy;
SplitType splitType;
List<Split> splits;
enum SplitType
{
EQUAL
UNEQU
PERCEN
udemy

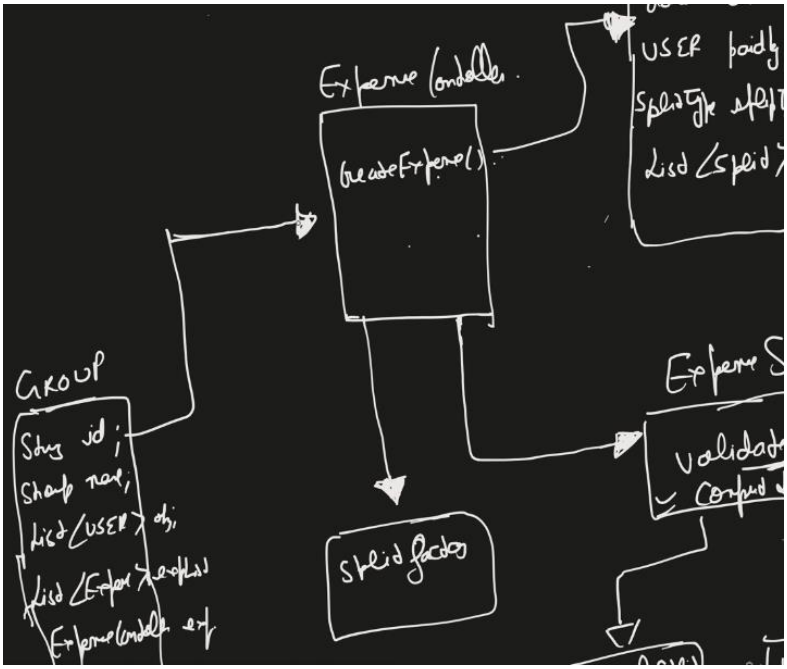

Expense.
String id;
String description;
double amount;
USER paidBy;
SplitType splitType;
List<Split> splits;
Expense Controller.
// Creating Expense.

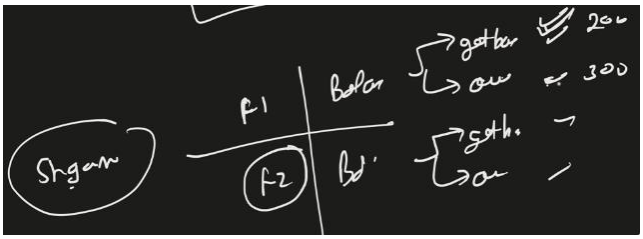

Expense Controller.
createExpense();
USER paidBy;
SplitType splitType;
List<Split> splits;
Split factory

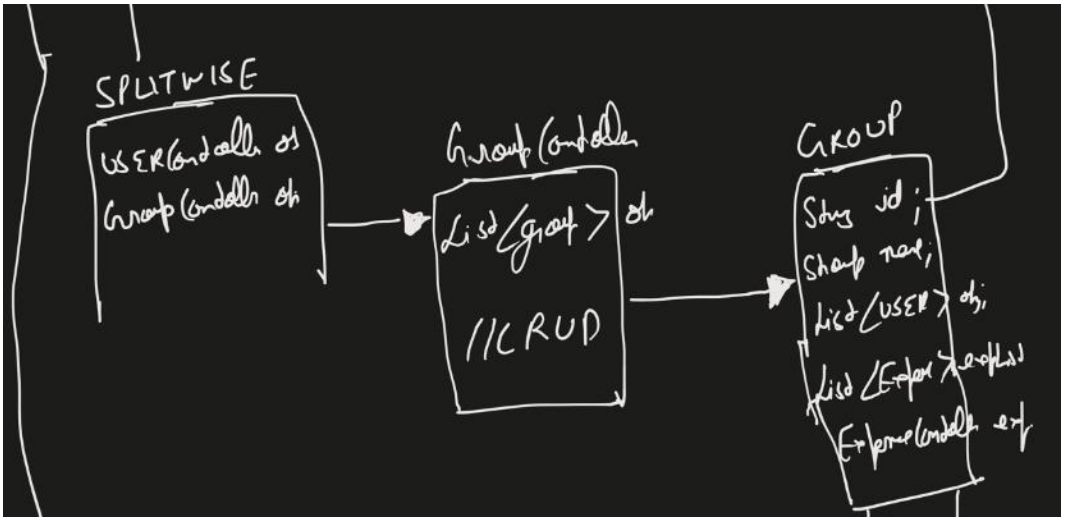

Expense Split
validateRequest();
Equal Split
unequal Split
percentage Split

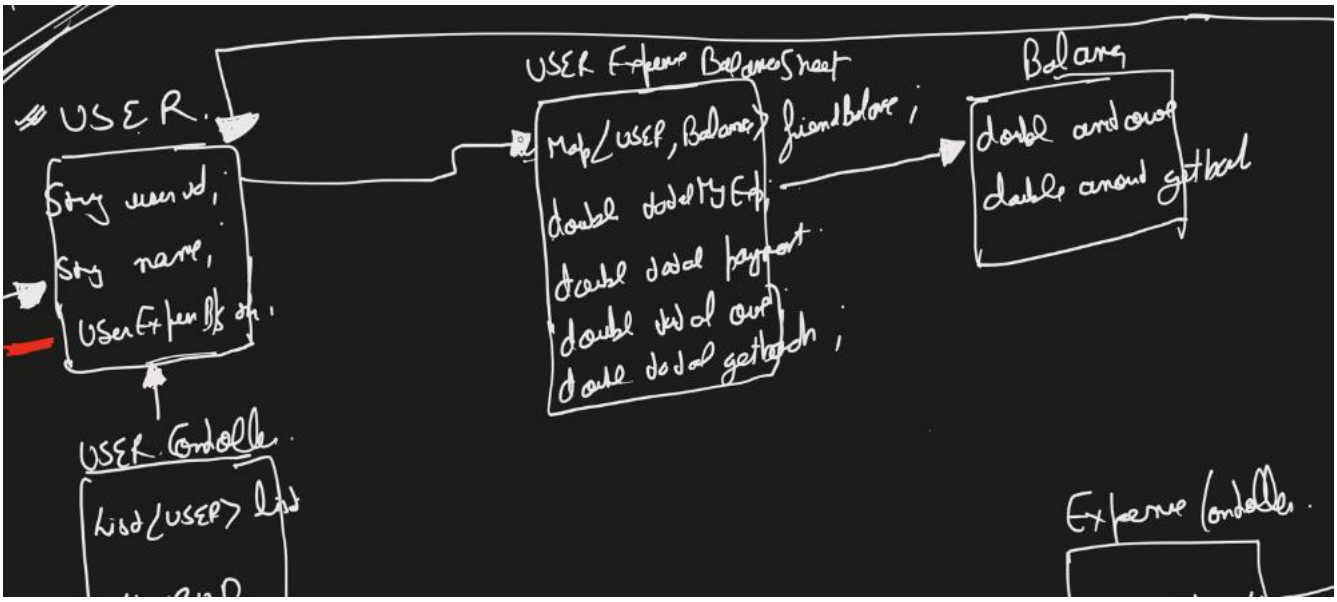

Expense Controller
USER paidBy;
SplitType splitType;
List<Split> splits;
CreateExpense()
easily extensible , can also add compute amount method to get amount for percentages etc
ExpenseSplit
ValidateRequest()
ComputeAmount()
SplitFactory
Expense
Amount ? 700
Paid ?
Split
f1 -100
f2 -200
f3 -300
Splits → Equal
Unequal
Expense
Amount ?
Paid ?
Split
f1 -100
f2 -200
f3 -300
Splits → Equal

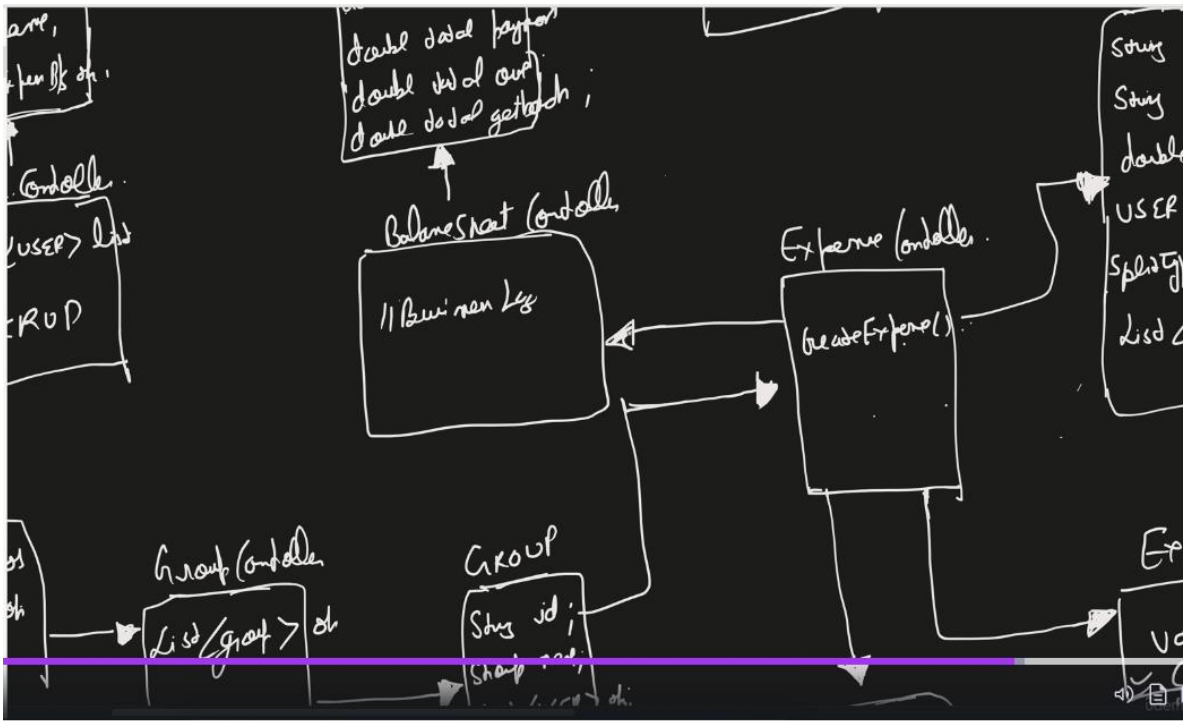

Shreyash
Total Amount owe -1000
getback +
Saurav f1 → 100
Sgitha f2 → 500
Saurav f3 → 400
Shreyas
UI
f1 getback owe
f2 get
f3
USER
String userId;
String name;
UserExpensesOth.
USER Controller
List<USER> list
//CRUD

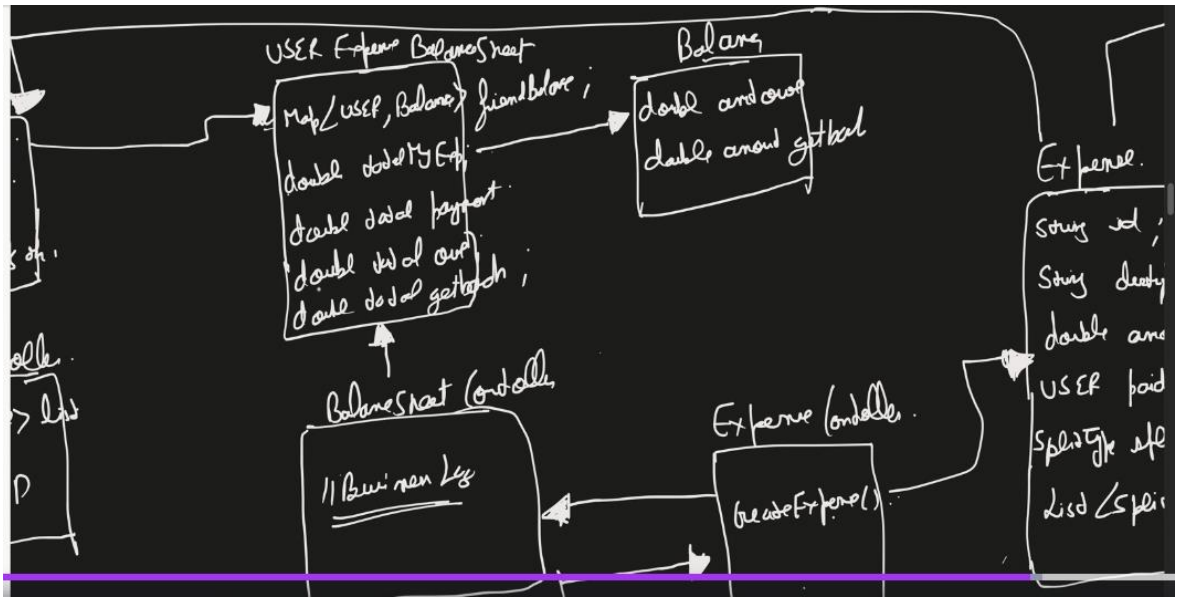

Expense Controller
GROUP
USER
Expense S
Validate
Split factor

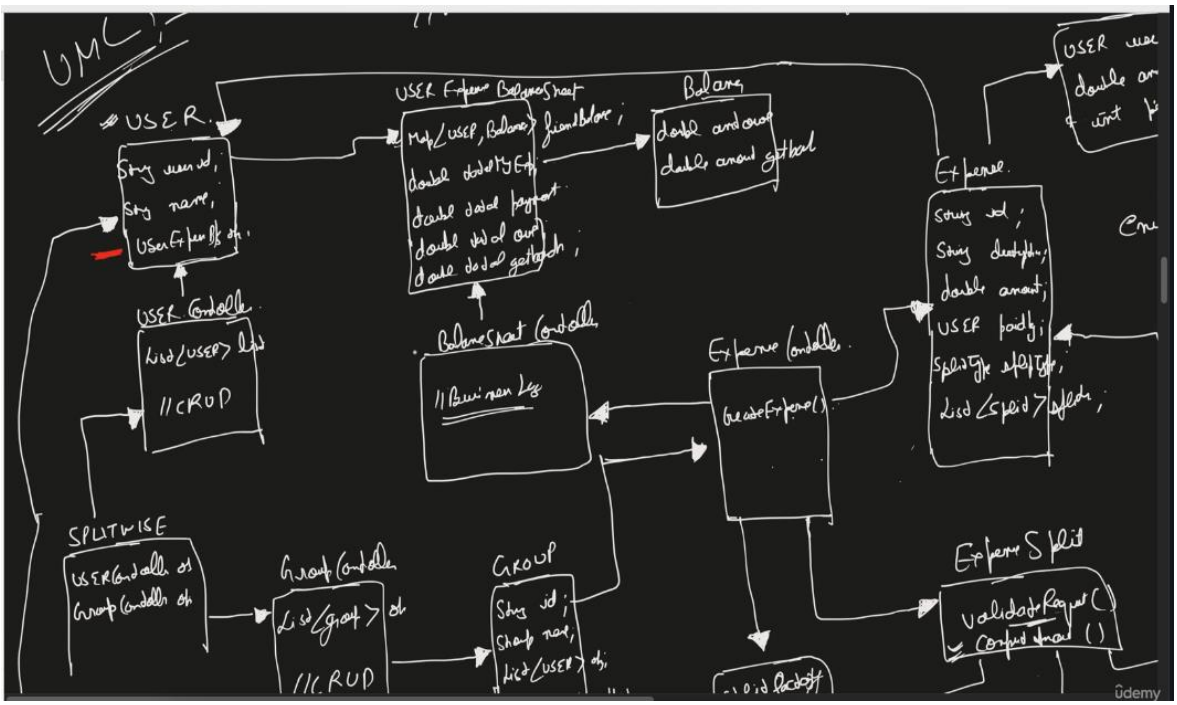

Sign in
F1
F2
Balance
Bal
get bal
owe
200
300

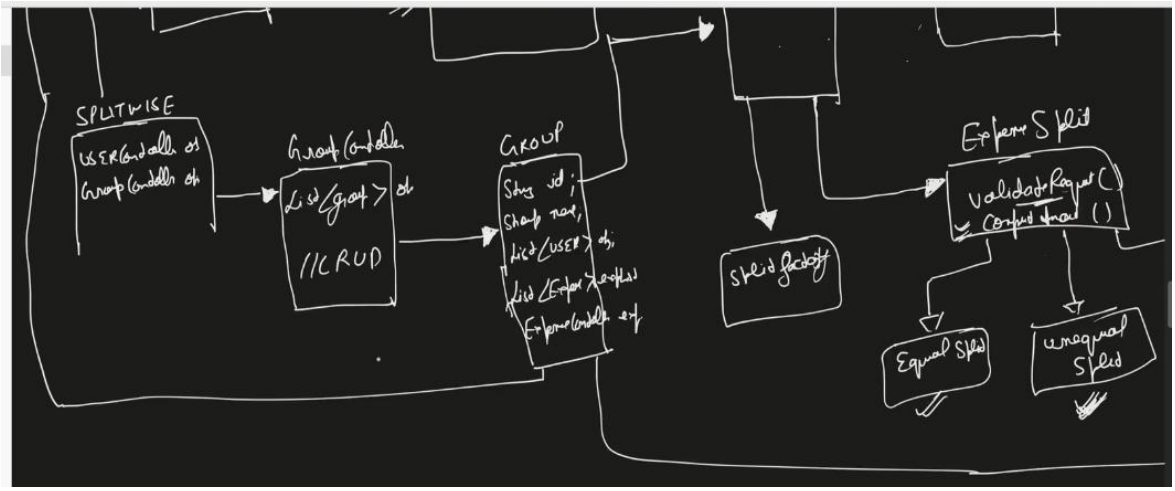

SPLITWISE
USER Controller
Group Controller
Group Controller
List<group>
CRUD
GROUP

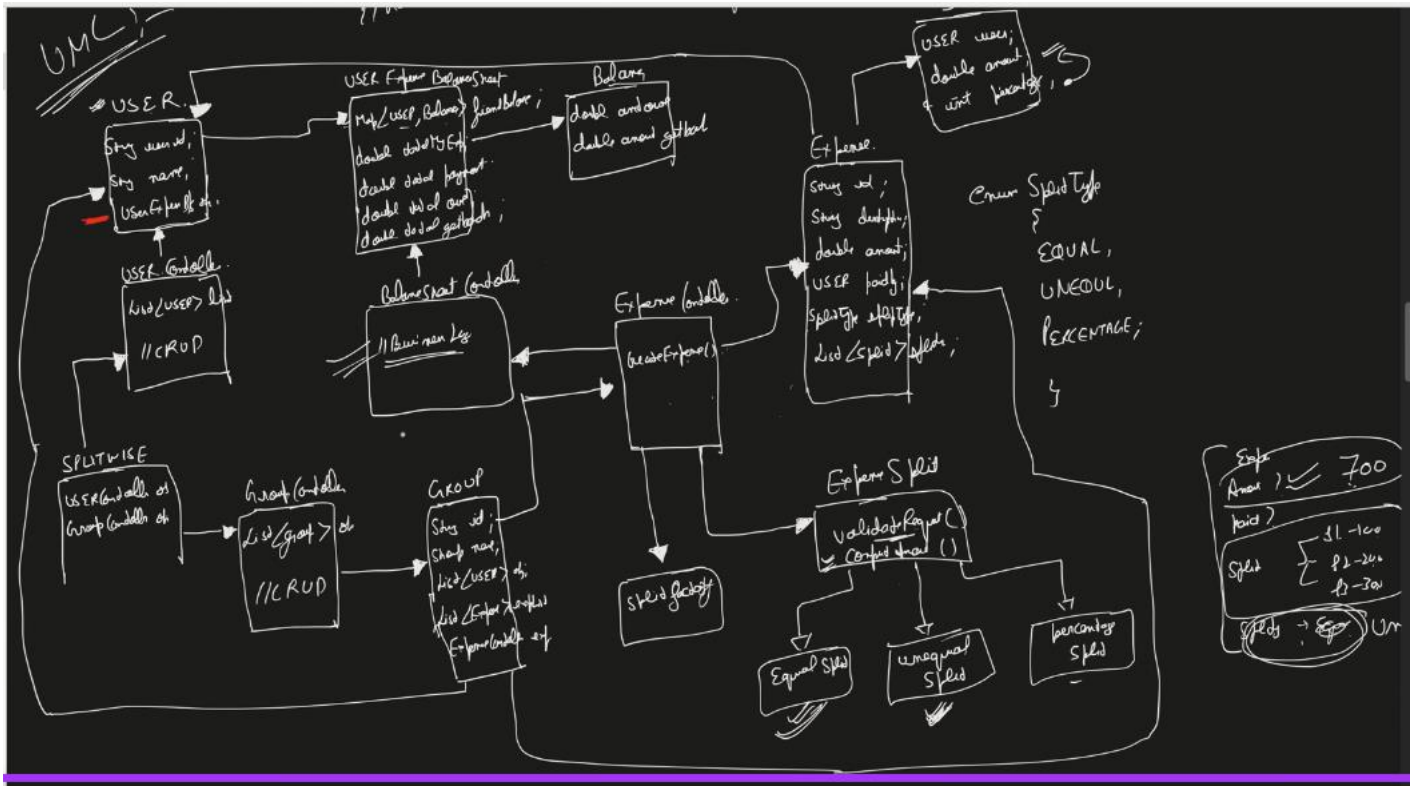

USER
USER Expense BalanceSheet
Balance
USER Controller
Expense Controller


BalanceSheet Controller
Expense Controller
Group Controller
GROUP


USER Expense BalanceSheet
Balance
Expense
BalanceSheet Controller
Expense Controller


UML
USER
USER Expense BalanceSheet
Balance
Expense
USER Controller
BalanceSheet Controller
Expense Controller
SPLITWISE
Group Controller
GROUP
Expense Split
//CRUD
udemy


SPLITWISE
Group Controller
GROUP
Expense Split
//CRUD
Split factory
Equal Split
unequal Split


UML
USER
USER Expense BalanceSheet
Balance
Expense
USER Controller
BalanceSheet Controller
Expense Controller
SPLITWISE
Group Controller
GROUP
Expense Split
//CRUD
Split factory
Equal Split
unequal Split
percentage Split
EQUAL,
UNEQUAL,
PERCENTAGE;

Simplify of Expense feature.

Algorithm graph

Balance sheet (Groups)

Total Expense: 250
Total Pg: 500
Total getback: = 250
Total owe: = 0

U1 → U2 getBack: 250 owe: 0
U1 → U3 getBack: 0 owe: 0
U1 → U4 getBack: 0 owe: 0

T.E = 250
T.P = 0
U2:
T.getba = 0
T.ow = 250

U2 → U1 getback: owe: 250
U2 → U3 getba: ow:
U2 → U7 get b: ow:

U1 $\xrightarrow{500\ Rs}$ SPLIT EQUALLY U1 & U2

Amt: 500
Split By: EQUALLY
Split detail: U1: 250, U2: 250
Paid By User1

Owing Group

TP: 900
Tot Ex: 300 700
T GB: 300 600
T.ow: 400

U1 → U2 getback: 300 ow: 400
U1 → U3 gB: 300 ow:

T Pg: 500
TE: 300 400
U2
T.ow = 300
Tgb = 400

U2 → U1 gB: 400 ow: 300
U2 → U3

U3 → U1 getB: ow: 300
U3 → U2
TE: 300
T.ow: 300

BFosd

900 Rs
EQUALLY
Paid By: User1.
→ U1: 300, U2: 300, U3: 300

500
UNEQUAL
Paid By: User2
→ U1: 400 Rs
U2: 100